

roots_poly.h

```

void laguer(VecComplex_I &a, Complex &x, Int &its) {
    Given the m+1 complex coefficients a[0..m] of the polynomial  $\sum_{i=0}^m a[i]x^i$ , and given a complex
    value x, this routine improves x by Laguerre's method until it converges, within the achievable
    roundoff limit, to a root of the given polynomial. The number of iterations taken is returned as
    its.

    const Int MR=8,MT=10,MAXIT=MT*MR;
    const Doub EPS=numeric_limits<Doub>::epsilon();
    Here EPS is the estimated fractional roundoff error. We try to break (rare) limit cycles with
    MR different fractional values, once every MT steps, for MAXIT total allowed iterations.
    static const Doub frac[MR+1]=
        {0.0,0.5,0.25,0.75,0.13,0.38,0.62,0.88,1.0};
    Fractions used to break a limit cycle.
    Complex dx,x1,b,d,f,g,h,sq,gp,gm,g2;
    Int m=a.size()-1;
    for (Int iter=1;iter<=MAXIT;iter++) { Loop over iterations up to allowed maximum.
        its=iter;
        b=a[m];
        Doub err=abs(b);
        d=f=0.0;
        Doub abx=abs(x);
        for (Int j=m-1;j>=0;j--) { Efficient computation of the polynomial and
            f=x*f+d; its first two derivatives. f stores  $P''/2$ .
            d=x*d+b;
            b=x*b+a[j];
            err=abs(b)+abx*err;
        }
        err *= EPS;
        Estimate of roundoff error in evaluating polynomial.
        if (abs(b) <= err) return; We are on the root.
        g=d/b; The generic case: Use Laguerre's formula.
        g2=g*g;
        h=g2-2.0*f/b;
        sq=sqrt(Doub(m-1)*(Doub(m)*h-g2));
        gp=g+sq;
        gm=g-sq;
        Doub abp=abs(gp);
        Doub abm=abs(gm);
        if (abp < abm) gp=gm;
        dx=MAX(abp,abm) > 0.0 ? Doub(m)/gp : polar(1+abx,Doub(iter));
        x1=x-dx;
        if (x == x1) return; Converged.
        if (iter % MT != 0) x=x1;
        else x -= frac[iter/MT]*dx;
        Every so often we take a fractional step, to break any limit cycle (itself a rare occur-
        rence).
    }
    throw("too many iterations in laguer");
    Very unusual; can occur only for complex roots. Try a different starting guess.
}

```

Here is a driver routine that calls laguer in succession for each root, performs the deflation, optionally polishes the roots by the same Laguerre method — if you are not going to polish in some other way — and finally sorts the roots by their real parts. (We will use this routine in Chapter 13.)

roots_poly.h

```

void zroots(VecComplex_I &a, VecComplex_0 &roots, const Bool &polish)
    Given the m+1 complex coefficients a[0..m] of the polynomial  $\sum_{i=0}^m a(i)x^i$ , this routine suc-
    cessively calls laguer and finds all m complex roots in roots[0..m-1]. The boolean variable
    polish should be input as true if polishing (also by Laguerre's method) is desired, false if the
    roots will be subsequently polished by other means.
    {
        const Doub EPS=1.0e-14; A small number.

```

```

Int i, its;
Complex x, b, c;
Int m=a.size()-1;
VecComplex ad(m+1);
for (Int j=0; j<=m; j++) ad[j]=a[j];
for (Int j=m-1; j>=0; j--) {
    x=0.0;
    VecComplex ad_v(j+2);
    for (Int jj=0; jj<j+2; jj++) ad_v[jj]=ad[jj];
    laguer(ad_v, x, its);
    if (abs(imag(x)) <= 2.0*EPS*abs(real(x)))
        x=Complex(real(x), 0.0);
    roots[j]=x;
    b=ad[j+1];
    for (Int jj=j; jj>=0; jj--) {
        c=ad[jj];
        ad[jj]=b;
        b=x*b+c;
    }
}
if (polish)
    for (Int j=0; j<=m; j++)
        laguer(a, roots[j], its);
for (Int j=1; j<=m; j++) {
    x=roots[j];
    for (i=j-1; i>=0; i--) {
        if (real(roots[i]) <= real(x)) break;
        roots[i+1]=roots[i];
    }
    roots[i+1]=x;
}
}

```

Copy of coefficients for successive deflation.
 Loop over each root to be found.
 Start at zero to favor convergence to smallest remaining root, and return the root.

Forward deflation.

Polish the roots using the undeflated coefficients.
 Sort roots by their real parts by straight insertion.

9.5.4 Eigenvalue Methods

The eigenvalues of a matrix $\mathbf{A}$ are the roots of the “characteristic polynomial” $P(x) = \det[\mathbf{A} - x\mathbf{I}]$. However, as we will see in Chapter 11, root finding is not generally an efficient way to find eigenvalues. Turning matters around, we can use the more efficient eigenvalue methods that are discussed in Chapter 11 to find the roots of arbitrary polynomials. You can easily verify (see, e.g., [6]) that the characteristic polynomial of the special $m \times m$ companion matrix

$$\mathbf{A} = \begin{pmatrix} -\frac{a_{m-1}}{a_m} & -\frac{a_{m-2}}{a_m} & \cdots & -\frac{a_1}{a_m} & -\frac{a_0}{a_m} \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix} \quad (9.5.12)$$

is equivalent to the general polynomial

$$P(x) = \sum_{i=0}^m a_i x^i \quad (9.5.13)$$

If the coefficients a_i are real, rather than complex, then the eigenvalues of $\mathbf{A}$ can be found using the routine `Unsymmeig` in §11.6 – §11.7 (see discussion there). This

method, implemented in the routine `zrhqr` following, is typically about a factor 2 slower than `zroots` (above). However, for some classes of polynomials, it is a more robust technique, largely because of the fairly sophisticated convergence methods embodied in `Unsymmeig`. If your polynomial has real coefficients, and you are having trouble with `zroots`, then `zrhqr` is a recommended alternative.

```
zrhqr.h void zrhqr(VecDoub_I &a, VecComplex_0 &rt)
Find all the roots of a polynomial with real coefficients,  $\sum_{i=0}^m a(i)x^i$ , given the coefficients
a[0..m]. The method is to construct an upper Hessenberg matrix whose eigenvalues are the
desired roots and then use the routine Unsymmeig. The roots are returned in the complex vector
rt[0..m-1], sorted in descending order by their real parts.
{
    Int m=a.size()-1;
    MatDoub hess(m,m);
    for (Int k=0;k<m;k++) {          Construct the matrix.
        hess[0][k] = -a[m-k-1]/a[m];
        for (Int j=1;j<m;j++) hess[j][k]=0.0;
        if (k != m-1) hess[k+1][k]=1.0;
    }
    Unsymmeig h(hess, false, true);    Find its eigenvalues.
    for (Int j=0;j<m;j++)
        rt[j]=h.wri[j];
}
```

9.5.5 Other Sure-Fire Techniques

The *Jenkins-Traub method* has become practically a standard in black-box polynomial root finders, e.g., in the IMSL library [3]. The method is too complicated to discuss here, but is detailed, with references to the primary literature, in [4].

The *Lehmer-Schur algorithm* is one of a class of methods that isolate roots in the complex plane by generalizing the notion of one-dimensional bracketing. It is possible to determine efficiently whether there are any polynomial roots within a circle of given center and radius. From then on it is a matter of bookkeeping to hunt down all the roots by a series of decisions regarding where to place new trial circles. Consult [1] for an introduction.

9.5.6 Techniques for Root Polishing

Newton-Raphson works very well for real roots once the neighborhood of a root has been identified. The polynomial and its derivative can be efficiently simultaneously evaluated as in §5.1. For a polynomial of degree n with coefficients $c[0] \dots c[n]$, the following segment of code carries out one cycle of Newton-Raphson:

```
p=c[n]*x+c[n-1];
p1=c[n];
for(i=n-2;i>=0;i--) {
    p1=p+p1*x;
    p=c[i]+p*x;
}
if (p1 == 0.0) throw("derivative should not vanish");
x -= p/p1;
```

Once all real roots of a polynomial have been polished, one must polish the complex roots, either directly or by looking for quadratic factors.

Direct polishing by Newton-Raphson is straightforward for complex roots if the above code is converted to complex data types. With real polynomial coefficients, note that your starting guess (tentative root) *must* be off the real axis, otherwise you will never get off that axis — and may get shot off to infinity by a minimum or maximum of the polynomial.

For real polynomials, the alternative means of polishing complex roots (or, for that matter, double real roots) is *Bairstow's method*, which seeks quadratic factors. The advantage of going after quadratic factors is that it avoids all complex arithmetic. Bairstow's method seeks a quadratic factor that embodies the two roots $x = a \pm ib$, namely

$$x^2 - 2ax + (a^2 + b^2) \equiv x^2 + Bx + C \quad (9.5.14)$$

In general, if we divide a polynomial by a quadratic factor, there will be a linear remainder

$$P(x) = (x^2 + Bx + C)Q(x) + Rx + S. \quad (9.5.15)$$

Given B and C , R and S can be readily found, by polynomial division (§5.1). We can consider R and S to be adjustable functions of B and C , and they will be zero if the quadratic factor is a divisor of $P(x)$.

In the neighborhood of a root, a first-order Taylor series expansion approximates the variation of R , S with respect to small changes in B , C :

$$R(B + \delta B, C + \delta C) \approx R(B, C) + \frac{\partial R}{\partial B}\delta B + \frac{\partial R}{\partial C}\delta C \quad (9.5.16)$$

$$S(B + \delta B, C + \delta C) \approx S(B, C) + \frac{\partial S}{\partial B}\delta B + \frac{\partial S}{\partial C}\delta C \quad (9.5.17)$$

To evaluate the partial derivatives, consider the derivative of (9.5.15) with respect to C . Since $P(x)$ is a fixed polynomial, it is independent of C , hence

$$0 = (x^2 + Bx + C)\frac{\partial Q}{\partial C} + Q(x) + \frac{\partial R}{\partial C}x + \frac{\partial S}{\partial C} \quad (9.5.18)$$

which can be rewritten as

$$-Q(x) = (x^2 + Bx + C)\frac{\partial Q}{\partial C} + \frac{\partial R}{\partial C}x + \frac{\partial S}{\partial C} \quad (9.5.19)$$

Similarly, $P(x)$ is independent of B , so differentiating (9.5.15) with respect to B gives

$$-xQ(x) = (x^2 + Bx + C)\frac{\partial Q}{\partial B} + \frac{\partial R}{\partial B}x + \frac{\partial S}{\partial B} \quad (9.5.20)$$

Now note that equation (9.5.19) matches equation (9.5.15) in form. Thus if we perform a second synthetic division of $P(x)$, i.e., a division of $Q(x)$ by the same quadratic factor, yielding a remainder $R_1x + S_1$, then

$$\frac{\partial R}{\partial C} = -R_1 \quad \frac{\partial S}{\partial C} = -S_1 \quad (9.5.21)$$

To get the remaining partial derivatives, evaluate equation (9.5.20) at the two roots of the quadratic, x_+ and x_- . Since

$$Q(x_{\pm}) = R_1x_{\pm} + S_1 \quad (9.5.22)$$

we get

$$\frac{\partial R}{\partial B}x_+ + \frac{\partial S}{\partial B} = -x_+(R_1x_+ + S_1) \quad (9.5.23)$$

$$\frac{\partial R}{\partial B}x_- + \frac{\partial S}{\partial B} = -x_-(R_1x_- + S_1) \quad (9.5.24)$$